

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételek, ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételek, ciklusok**
 - Röppálya számítás**
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Röppálya határértékeinek számítása [1/2]

roppalya.py (v1.0) [1-17]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Ferdehajítás, röppálya, szélsőértékeinek számolása
9"""
10from math import pi
11from math import sin
12from math import cos
13
14G = 9.80665
15
16def deg_to_rad(alfa):
17    return alfa * pi / 180.0
```

Röppálya határértékeinek számítása [2/2]

roppalya.py (v1.0) [19–34]

```
19 def main():
20     v0 = float(input("Kezdősebesség(m/s)?_"))
21     alfa = float(input("Szög(fok)?_"))
22     if 0.0 <= alfa <= 90.0 and v0 >= 0.0:
23         vy = v0 * sin(deg_to_rad(alfa))
24         t = vy / G
25         sy = G / 2.0 * t ** 2
26         print(f"A_röppálya_legnagyobb_magasága:{sy:.3f}")
27         vx = v0 * cos(deg_to_rad(alfa))
28         sx = vx * 2.0 * t
29         print(f"A_tárgy_távolsága_földetéréskor:{sx:.3f}")
30     else:
31         print("Hibás_adatok!")
32
33 if __name__ == "__main__":
34     main()
```

Röppálya határértékeinek számítása

A probléma

- Eldobunk egy testet, milyen magasra megy, és hová esik?
- Csak egyszerően: sík terepen, légellenállás nélkül, pontszerű testtel, közvetlenül a felszínről dobjuk.



Röppálya határértékeinek számítása

roppalya.py (v0.1) [1–14]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_előadás_példaprogram
4SZTE_TTIK, _Informatikai_Intézet, _Szoftverfejlesztés_Tanszék
5Gergely_Tamás, _gertom@inf.u-szeged.hu, _2024-07-15
6cc-by-nc-nd
7
8Ferdehajítás_roppálya_szélsőértékeinek_számolása
9"""
10def main():
11    pass
12
13if __name__ == "__main__":
14    main()
```

Röppálya határértékeinek számítása

Az elmélet

- Egy kis fizika
 - google search, ha nincs meg középiskolából
https://hu.wikipedia.org/wiki/Ferde_haj%C3%ADt%C3%A1s
 - v_0 : kezdősebesség; α : kilövési szög; g : nehézségi gyorsulás
 - $v_x(t) = v_0 \cdot \cos \alpha$; a vízszintes irányú sebesség a t időpillanatban (nem függ t -től)
 - $v_y(t) = v_0 \cdot \sin \alpha - g \cdot t$; a függőleges irányú sebesség a t időpillanatban
 - $x(t) = v_x(0) \cdot t$; a vízszintes irányú elmozdulás a t időpillanatban
 - $y(t) = v_y(0) \cdot t - \frac{g}{2} \cdot t^2$; a függőleges irányú elmozdulás a t időpillanatban
 - $t_m = \frac{v_y(0)}{g}$; legnagyobb magasság pillanata ($v_y(t_m) = 0$)
 - $t_t = 2 \frac{v_y(0)}{g}$; teljes repülési idő ($y(t_t) = 0$ és $t_t \neq 0$)
- A kérdések:
 - Milyen magasra megy? $y(t_m) = ?$
 - Milyen messzire megy? $x(t_t) = ?$
- Számoljunk fixen m/s -ban.

Matematikai függvények és konstansok

A `math` modul [6]

- Egyes matematikai függvények és konstansok, mint például a `sin`, `cos`, `log` vagy az `e` és a `pi` nem részei a nyelvnek, de egy standard Python modulban össze vannak gyűjtve.
- Használatukhoz a teljes `math` modul `import`-jára, vagy a használni kívánt konkrét elemek `math` modulból való `import`-jára lesz szükség:

```
import math

math.sin(0.0)
math.cos(math.pi)
```

```
from math import log
from math import e

log(e)
```

Röppálya határértékeinek számítása

Kezdősebességek

- Először számoljuk ki a vízszintes és függőleges irányú kezdősebességeket.
- Egyelőre dolgozzunk fix adatokkal.



Röppálya határértékeinek számítása

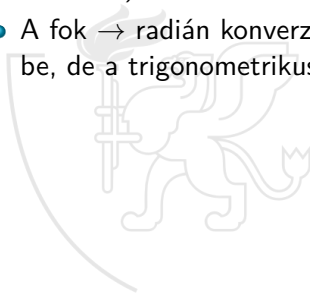
roppalya.py (v0.2) [1–23]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-07-15
6cc-by-nc-nd
7
8Ferdehajítás röppálya szélsőértékeinek számolása
9"""
10from math import pi
11from math import sin
12from math import cos
13
14def main():
15    v0 = 10.0                                # TESZT
16    alfa = 30.0                              # TESZT
17    vy = v0 * sin(alfa * pi / 180.0)
18    print(f"Függőleges kezdősebesség (vy): {vy} m/s")    # TESZT
19    vx = v0 * cos(alfa * pi / 180.0)
20    print(f"Vízszintes kezdősebesség (vx): {vx} m/s")    # TESZT
21
22if __name__ == "__main__":
23    main()
```

Röppálya határértékeinek számítása

Idő

- Számítsuk ki a repülési időt (a teljeset, vagy a maximális magasság elérésének idejét, teljesen mindegy; a 2-es szorzót majd korrigáljuk, ahol kell).
- Szükségünk lesz a gravitációs gyorsulás értékére, de a tesztelés kedvéért egyelőre válasszunk valami emészthető értéket (amivel az ellenőrzéskor fejben könnyen tudunk számolni).
- A fok $\rightarrow$ radián konverziót pedig emeljük ki külön függvénybe (a programunk fokokat kér be, de a trigonometrikus függvények radiánt használnak).



Röppálya határértékeinek számítása

roppalya.py (v0.3) [10–30]

```
10 from math import pi
11 from math import sin
12 from math import cos
13
14 G = 10.0
15
16 def deg_to_rad(alfa):
17     return alfa * pi / 180.0
18
19 def main():
20     v0 = 10.0                                # TESZT
21     alfa = 30.0                              # TESZT
22     vy = v0 * sin(deg_to_rad(alfa))
23     print(f"Függőleges kezdősebesség (vy): {vy} m/s")    # TESZT
24     t = vy / G
25     print(f"Repülési idő a csúcspontig: {t} s")          # TESZT
26     vx = v0 * cos(deg_to_rad(alfa))
27     print(f"Vízszintes kezdősebesség (vx): {vx} m/s")    # TESZT
28
29 if __name__ == "__main__":
30     main()
```

Röppálya határértékeinek számítása

Tesztelés

- És mi van akkor, ha lefelé hajítunk?
- A kritikus gondolkodás, a probléma(megoldás) határainak felismerése nagyon fontos!



Röppálya határértékeinek számítása

roppalya.py (v0.4) [10–30]

```
10 from math import pi
11 from math import sin
12 from math import cos
13
14 G = 10.0
15
16 def deg_to_rad(alfa):
17     return alfa * pi / 180.0
18
19 def main():
20     v0 = 10.0                                # TESZT
21     alfa = -30.0                             # TESZT
22     vy = v0 * sin(deg_to_rad(alfa))
23     print(f"Függőleges kezdősebesség: {vy} m/s")    # TESZT
24     t = vy / G
25     print(f"Repülési idő a csúcspontig: {t} s")    # TESZT
26     vx = v0 * cos(deg_to_rad(alfa))
27     print(f"Vízszintes kezdősebesség: {vx} m/s")    # TESZT
28
29 if __name__ == "__main__":
30     main()
```

bool (logikai) értékek műveletei

Logikai műveletek

- egyoperandusú műveletek
 not tagadás, negáció
- kétoperandusú műveletek
 and logikai és
 or logikai vagy

```
not a  
a and b  
a or b
```



bool (logikai) kifejezések

Az and és or működése Python-ban

- A logikai műveletek kiértékelése mindig a rövidített kiértékelés szerint történik, vagyis
 - Az `A or B` kifejezés rövidített kiértékelése során először kiértékelődik a `bool(A)` kifejezés, ha ennek értéke `True`, akkor a teljes kifejezés eredménye `A` lesz (a `B` tag kiértékelése elmarad); ha `bool(A)` értéke `False`, akkor a kifejezés eredménye `B` értéke lesz.
 - Az `A and B` kifejezés rövidített kiértékelése során először kiértékelődik a `bool(A)` kifejezés, ha ennek értéke `False`, akkor a teljes kifejezés eredménye `A` lesz (a `B` tag kiértékelése elmarad); ha `bool(A)` értéke `True`, akkor a kifejezés eredménye `B` értéke lesz.
- Ha `A` és `B` logikai értékek, akkor ez a logika a következőképpen egyszerűsödik:
 - Az `A or B` kifejezés rövidített kiértékelése során először kiértékelődik az `A` kifejezés, ha ennek értéke `True`, akkor a teljes kifejezés eredménye `True` lesz (a `B` tag kiértékelése elmarad); ha `A` értéke `False`, akkor a kifejezés eredménye `B` értéke lesz.
 - Az `A and B` kifejezés rövidített kiértékelése során először kiértékelődik az `A` kifejezés, ha ennek értéke `False`, akkor a teljes kifejezés eredménye `False` lesz (a `B` tag kiértékelése elmarad); ha `A` értéke `True`, akkor a kifejezés eredménye `B` értéke lesz.

Az if utasítás

- Ha valamilyen feltétel alapján egyik vagy másik utasítást akarjuk végrehajtani.
 - Végre kell-e hajtani valamit?
 - Számoljuk ki f abszolút értékét!

```
if f < 0:  
    f = -f
```

- Két utasítás közül az egyiket hajtsuk végre!
 - a és b közül melyik a kisebb érték?

```
if a <= b:  
    k = a  
else:  
    k = b
```

Röppálya határértékeinek számítása

roppalya.py (v0.5) [10–33]

```
10 from math import pi
11 from math import sin
12 from math import cos
13
14 G = 10.0
15
16 def deg_to_rad(alfa):
17     return alfa * pi / 180.0
18
19 def main():
20     v0 = 10.0                                # TESZT
21     alfa = -30.0                             # TESZT
22     if 0.0 <= alfa <= 90.0 and v0 >= 0.0:
23         vy = v0 * sin(deg_to_rad(alfa))
24         print(f"Függőleges kezdősebesség: {vy} m/s") # TESZT
25         t = vy / G
26         print(f"Repülési idő a csúcspontig: {t} s")   # TESZT
27         vx = v0 * cos(deg_to_rad(alfa))
28         print(f"Vízszintes kezdősebesség: {vx} m/s") # TESZT
29     else:
30         print("Hibás adatok!")
31
32 if __name__ == "__main__":
33     main()
```

Röppálya határértékeinek számítása

Fejlesztés

- Számoljuk és írassuk ki a kívánt értékeket milliméter (három tizedesjegy) pontossággal.



Röppálya határértékeinek számítása

roppalya.py (v0.6) [10–34]

```
10 from math import pi
11 from math import sin
12 from math import cos
13
14 G = 10.0
15
16 def deg_to_rad(alfa):
17     return alfa * pi / 180.0
18
19 def main():
20     v0 = 10.0                                # TESZT
21     alfa = 45.0                              # TESZT
22     if 0.0 <= alfa <= 90.0 and v0 >= 0.0:
23         vy = v0 * sin(deg_to_rad(alfa))
24         t = vy / G
25         sy = G / 2.0 * t ** 2
26         print(f"A röppálya legnagyobb magasága: {sy:.3f}")
27         vx = v0 * cos(deg_to_rad(alfa))
28         sx = vx * 2.0 * t
29         print(f"A tárgy távolsága földetéréskor: {sx:.3f}")
30     else:
31         print("Hibás adatok!")
32
33 if __name__ == "__main__":
34     main()
```

Röppálya határértékeinek számítása

Fejlesztés

- Végül, térjünk át felhasználói inputra, és használjunk valós gravitációs gyorsulási értéket (nem feledve, hogy m/s^2 -ben kell megadni).



Röppálya határértékeinek számítása [1/2]

roppalya.py (v1.0) [1-17]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Ferdehajítás, röppálya, szélsőértékeinek számolása
9"""
10from math import pi
11from math import sin
12from math import cos
13
14G = 9.80665
15
16def deg_to_rad(alfa):
17    return alfa * pi / 180.0
```

Röppálya határértékeinek számítása [2/2]

roppalya.py (v1.0) [19–34]

```
19 def main():
20     v0 = float(input("Kezdősebesség(m/s)?_"))
21     alfa = float(input("Szög(fok)?_"))
22     if 0.0 <= alfa <= 90.0 and v0 >= 0.0:
23         vy = v0 * sin(deg_to_rad(alfa))
24         t = vy / G
25         sy = G / 2.0 * t ** 2
26         print(f"A_röppálya_legnagyobb_magasága:{sy:.3f}")
27         vx = v0 * cos(deg_to_rad(alfa))
28         sx = vx * 2.0 * t
29         print(f"A_tárgy_távolsága_földetéréskor:{sx:.3f}")
30     else:
31         print("Hibás_adatok!")
32
33 if __name__ == "__main__":
34     main()
```

Tartalomjegyzék

- 
- 1 Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételek, ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció**
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Röppálya szimulációja [1/2]

roppalya_szimulacio.py (v1.0) [1–13]

```
1#!/usr/bin/env python3
2"""
3Programozás┐Alapjai┐előadás┐példaprogram
4SZTE┐TTIK,┐Informatikai┐Intézet,┐Szoftverfejlesztés┐Tanszék
5Gergely┐Tamás,┐gertom@inf.u-szeged.hu,┐2024-07-16
6cc-by-nc-nd
7
8Ferdehajítás┐röppálya┐szimulációja
9"""
10from math import sin
11from math import cos
12from roppalya import G
13from roppalya import deg_to_rad
```

Röppálya szimulációja [2/2]

roppalya_szimulacio.py (v1.0) [15–30]

```
15 def main():
16     v0 = float(input("Kezdősebesség(m/s)? "))
17     alfa = float(input("Szög(fok)? "))
18     dt = float(input("Delta_t(s)? "))
19     t = 0.0
20     vx, vy = v0 * cos(deg_to_rad(alfa)), v0 * sin(deg_to_rad(alfa))
21     sx, sy = 0.0, 0.0
22     while sy >= 0.0:
23         print(f"{t:.3f}s: p({sx:.3f}, {sy:.3f}); v={vx:.3f}, {vy:.3f}")
24         sx += vx * dt
25         sy += vy * dt - G / 2.0 * dt ** 2
26         vy -= G * dt
27         t += dt
28
29 if __name__ == "__main__":
30     main()
```

Röppálya szimulációja

A probléma

- Eldobunk egy testet, milyen röppályán mozog?
- Csak egyszerűen: sík terepen, légellenállás nélkül, pontszerű testtel, közvetlenül a felszínről dobjuk.
- Egyenletes időközönként figyeljük, hogy hol van a test.
- Emlékszünk még?
 - $v_{0,x} = v_0 \cdot \cos \alpha$
 - $v_{0,y} = v_0 \cdot \sin \alpha$
 - $\Delta s_x = v_x \cdot \Delta t$
 - $\Delta s_y = v_y \cdot \Delta t \left(+ \frac{g}{2} \cdot \Delta t^2 \right)$
 - $\Delta v_x = 0$
 - $\Delta v_y = g \cdot \Delta t$
- Számoljunk fixen m/s -ban.

Röppálya szimulációja

roppalya_szimulacio.py (v0.2) [1–19]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-07-16
6cc-by-nc-nd
7
8Ferdehajítás, röppálya, szimulációja
9"""
10def main():
11    v0 = float(input("Kezdősebesség (m/s)? "))
12    alfa = float(input("Szög (fok)? "))
13    dt = float(input("Delta t (s)? "))
14    print(f"Kezdősebesség: {v0:.3f} m/s")          # TESZT
15    print(f"Kilövési szög: {alfa:.0f} fok")        # TESZT
16    print(f"Időszeglet hossza: {dt:.3f} s")        # TESZT
17
18if __name__ == "__main__":
19    main()
```

Röppálya szimulációja

Kezdősebesség-komponensek

- Először számoljuk ki a kezdősebesség vízszintes és függőleges komponenseit.
- A szög $\rightarrow$ radián konverziót egyszer már megírtuk, `deg_to_rad` névvel ott van a `roppalya.py` utolsó verziójában. Hát használjuk onnan!



Röppálya szimulációja

roppalya_szimulacio.py (v0.3) [1–22]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-07-16
6cc-by-nc-nd
7
8Ferdehajítás röppálya szimulációja
9"""
10from math import sin
11from math import cos
12from roppalya import deg_to_rad
13
14def main():
15    v0 = float(input("Kezdősebesség (m/s)? "))
16    alfa = float(input("Szög (fok)? "))
17    dt = float(input("Delta t (s)? "))
18    vx, vy = v0 * cos(deg_to_rad(alfa)), v0 * sin(deg_to_rad(alfa))
19    print(f"vx={vx:.3f} m/s; vy={vy:.3f} m/s") # TESZT
20
21if __name__ == "__main__":
22    main()
```

A Python értékadó utasítások

- Python-ban a

```
v = v ⊙ e
```

alakú értékadásokat sok esetben

```
v ⊙ = e
```

alakban rövidíthetjük, ahol a $\odot$ tetszőleges műveletet jelölhet.

```
>>> x = 5
```

```
>>> x
```

```
5
```

```
>>> x += 2
```

```
>>> x
```

```
7
```

```
>>> x *= 6
```

```
>>> x
```

```
42
```

```
>>> x -= 10
```

```
>>> x
```

```
32
```

```
>>> x **= 2
```

```
>>> x
```

```
1024
```

```
>>> x /= 100
```

```
>>> x
```

```
10.24
```

```
>>> x %= 4
```

```
>>> x
```

```
2.24
```

```
>>> x //= 0.5
```

```
>>> x
```

```
4.0
```

A while utasítás

- Ha valamilyen műveletet mindaddig végre kell hajtani, amíg egy feltétel igaz a `while` utasítás használható.
 - A művelet végrehajtása nem szükséges a feltétel kiértékeléséhez.
 - A feltétel ellenőrzése a művelet előtt történik, így ha a feltétel kezdetben hamis volt, a műveletet egyszer sem hajtjuk végre.

```
while sum_of_hand < 19:  
    sum_of_hand += draw_new_card()
```



Röppálya szimulációja

Sebességadatok a repülés során

- Számoljuk ki a maximális repülési időt.
- Számoljuk ki az aktuális sebességet a repülés minden Δt időpontjához.



Röppálya szimulációja

roppalya_szimulacio.py (v0.4) [10–28]

```
10 from math import sin
11 from math import cos
12 from roppalya import G
13 from roppalya import deg_to_rad
14
15 def main():
16     v0 = float(input("Kezdősebesség (m/s)? "))
17     alfa = float(input("Szög (fok)? "))
18     dt = float(input("Delta t (s)? "))
19     t = 0.0
20     vx, vy = v0 * cos(deg_to_rad(alfa)), v0 * sin(deg_to_rad(alfa))
21     tmax = 2.0 * vy / G
22     while t <= tmax:
23         print(f"{t:.3f}s: vx={vx:.3f}, vy={vy:.3f}")
24         vy -= G * dt
25         t += dt
26
27 if __name__ == "__main__":
28     main()
```

Röppálya szimulációja

Kezdősebesség-komponensek

- Számoljunk helyadatokat is.
- A mozgást két „számolt” pillanat között tekintsük egyenes vonalú egyenletes sebességű mozgásnak.



Röppálya szimulációja [1/2]

roppalya_szimulacio.py (v0.5) [1-13]

```
1#!/usr/bin/env python3
2"""
3Programozás┐Alapjai┐előadás┐példaprogram
4SZTE┐TTIK,┐Informatikai┐Intézet,┐Szoftverfejlesztés┐Tanszék
5Gergely┐Tamás,┐gertom@inf.u-szeged.hu,┐2024-07-16
6cc-by-nc-nd
7
8Ferdehajítás┐röppálya┐szimulációja
9"""
10from math import sin
11from math import cos
12from roppalya import G
13from roppalya import deg_to_rad
```

Röppálya szimulációja [2/2]

roppalya_szimulacio.py (v0.5) [15–31]

```
15 def main():
16     v0 = float(input("Kezdősebesség(m/s)? "))
17     alfa = float(input("Szög(fok)? "))
18     dt = float(input("Delta_t(s)? "))
19     t = 0.0
20     vx, vy = v0 * cos(deg_to_rad(alfa)), v0 * sin(deg_to_rad(alfa))
21     sx, sy = 0.0, 0.0
22     tmax = 2.0 * vy / G
23     while t <= tmax:
24         print(f"{t:.3f}s: p({sx:.3f}, {sy:.3f}); v={vx:.3f}, {vy:.3f}")
25         sx += vx * dt
26         sy += vy * dt
27         vy -= G * dt
28         t += dt
29
30 if __name__ == "__main__":
31     main()
```

- Felesleges előre kiszámolni a repülési időt, elég, ha a tárgy aktuális magasságát figyeljük.



Röppálya szimulációja [1/2]

roppalya_szimulacio.py (v0.6) [1–14]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_előadás_példaprogram
4SZTE_TTIK, Informatikai_Intézet, Szoftverfejlesztés_Tanszék
5Gergely_Tamás, gertom@inf.u-szeged.hu, 2024-07-16
6cc-by-nc-nd
7
8Ferdehajítás_röppálya_szimulációja
9"""
10from math import sin
11from math import cos
12from roppalya import G
13from roppalya import deg_to_rad
```

Röppálya szimulációja [2/2]

roppalya_szimulacio.py (v0.6) [16–31]

```
16 v0 = float(input("Kezdősebesség(m/s)?"))
17 alfa = float(input("Szög(fok)?"))
18 dt = float(input("Delta_t(s)?"))
19 t = 0.0
20 vx, vy = v0 * cos(deg_to_rad(alfa)), v0 * sin(deg_to_rad(alfa))
21 sx, sy = 0.0, 0.0
22 while sy >= 0.0:
23     print(f"{t:.3f}s: p({sx:.3f}, {sy:.3f}); v={vx:.3f}, {vy:.3f}")
24     sx += vx * dt
25     sy += vy * dt
26     vy -= G * dt
27     t += dt
28
29 if __name__ == "__main__":
30     main()
```

- Tegyük kicsit pontosabbá a számítást (a helyadatnál egyenletes helyett egyenletesen gyorsuló mozgással számoljunk).



Röppálya szimulációja [1/2]

roppalya_szimulacio.py (v1.0) [1–13]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-07-16
6cc-by-nc-nd
7
8Ferdehajítás röppálya szimulációja
9"""
10from math import sin
11from math import cos
12from roppalya import G
13from roppalya import deg_to_rad
```

Röppálya szimulációja [2/2]

roppalya_szimulacio.py (v1.0) [15–30]

```
15 def main():
16     v0 = float(input("Kezdősebesség(m/s)? "))
17     alfa = float(input("Szög(fok)? "))
18     dt = float(input("Delta t(s)? "))
19     t = 0.0
20     vx, vy = v0 * cos(deg_to_rad(alfa)), v0 * sin(deg_to_rad(alfa))
21     sx, sy = 0.0, 0.0
22     while sy >= 0.0:
23         print(f"{t:.3f}s: p({sx:.3f}, {sy:.3f}); v={vx:.3f}, {vy:.3f}")
24         sx += vx * dt
25         sy += vy * dt - G / 2.0 * dt ** 2
26         vy -= G * dt
27         t += dt
28
29 if __name__ == "__main__":
30     main()
```